

COMP 2522 L07 Quiz

Mayvee, Aika, Mischa, Grace

- 1. Explain what a method reference is. List the four types of method references and give a short example of each.**

Example Answer:

A method reference is a shorthand syntax used instead of lambda expressions to call an existing method using the :: operator.

The 4 types are:

- Static method
 - Ex. MathUtils::add
- Instance method of an object
 - Ex. printer::printMessage
- Instance method of an object of a certain type
 - Ex. String::compareTo
- Constructor reference
 - Ex. Car::new

- 2. What are generics in Java? Give two advantages of using generics instead of Object.**

Example Answer:

Generics let us write classes and methods that work with different data types, but still keep type checking at compile time. Instead of storing everything as an Object, we can specify the type we expect.

Advantages:

- Generics prevent many runtime errors. For example, if we try to put the wrong type into a generic class, the compiler will catch it right away instead of crashing later.
- We don't need to do type casting when retrieving values. This makes the code cleaner and easier to read, and it also reduces mistakes.

- 3. Explain the difference between ? extends T and ? super T. When should each be used?**

Example Answer:

? extends T is used when we want a generic type to accept objects of type T or any subclass of T, but we only need to read values from it. The reason we cannot add elements is because the exact type is unknown at compile time. For example, if a method only needs to use objects as type T and does not modify them, using ? extends T makes the method more flexible and safe. This is often used when a generic structure or parameter is acting as a producer of data.

? super T is used when we want a generic type to accept objects of type T or any superclass of T. This allows us to safely insert values of type T or its subclasses. However, when reading values, the compiler can only guarantee that the result is of type Object. This is useful when a method needs to write or insert values into a generic structure.

4. What is the purpose of nested or inner classes in Java? Name the four types.

Example Answer:

Nested classes are classes defined inside another class. They are useful for organizing code and keeping related functionality together. They also help improve encapsulation because the inner class can access private members of the outer class.

The four types are:

- Static nested classes
- Non-static inner classes
- Local inner classes
- Anonymous inner classes

5. A company wants a reusable class to store employee IDs of different types (String, Integer, etc.).

Create a generic class called Storage<T> with:

- A private field
- A setter
- A getter

Explain how this class prevents runtime errors compared to using Object.

Example Answer:

```
public class Storage<T> {  
  
    private T value;  
  
    public void setValue(final T value) {
```

```

        this.value = value;
    }

    public T getValue() {

        final T result;
        result = value;

        return result;
    }
}

```

This class prevents runtime errors because the type is checked at compile time. When we create something like `Storage<String>`, the compiler makes sure only `String` values can be stored. If we try to store an `Integer`, it will give a compile-time error.

If we used an `Object` instead, any type could be stored, and we would need to cast the value when retrieving it. This could cause a `ClassCastException` at runtime if the wrong type was stored. Generics avoid this problem because the type is enforced before the program runs.

6. A university system wants to keep student academic records private. The developer decides to use an inner class to manage grades because it should only be accessed through the Student class.

Write a class that:

- **Uses a non static inner class to manage grades.**
- **Demonstrates how the inner class can access private fields of the outer class.**
- **Shows how the outer class creates and uses the inner class.**

Example Answer:

```

class Student {

    private String name;
    private int studentId;

    public Student(final String name,
                   final int studentId) {

        this.name = name;
        this.studentId = studentId;
    }
}

```

```
class GradeRecord {  
  
    private double grade;  
  
    public GradeRecord(final double grade) {  
  
        this.grade = grade;  
    }  
  
    public void displayRecord() {  
  
        System.out.println("Student: " + name);  
        System.out.println("ID: " + studentId);  
        System.out.println("Grade: " + grade);  
    }  
}  
  
    public void addGrade(final double grade) {  
  
        final GradeRecord record;  
  
        record = new GradeRecord(grade);  
        record.displayRecord();  
    }  
}
```